САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Дисциплина: Бэк-энд разработка

Отчет

Лабораторная работа №1

Выполнил:

Максимова Ольга

Группа K3342

Проверил:
Добряков Д. И.

Санкт-Петербург

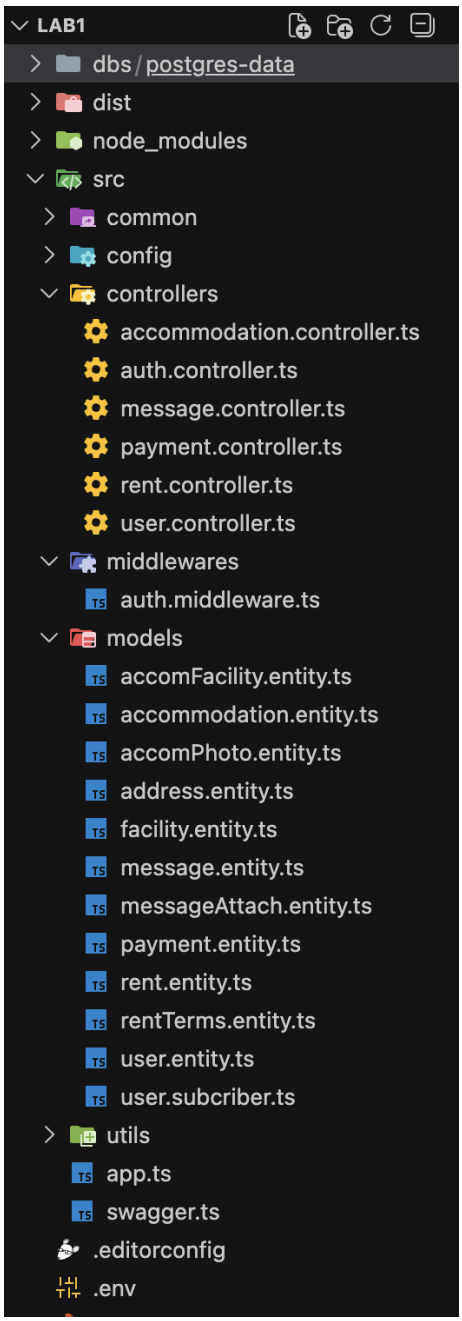
2026 г.

Задание:

1. Используйте любой boilerplate, который считаете наиболее подходящим для ваших нужд
2. С учетом имеющихся данных по результатам ДЗ1 и ДЗ2 реализуйте:
    - модели
    - представления
    - контроллеры
3. В результате выполнения лабораторной работы вы должны продемонстрировать полностью работающее API согласно вашего варианта

Ход работы

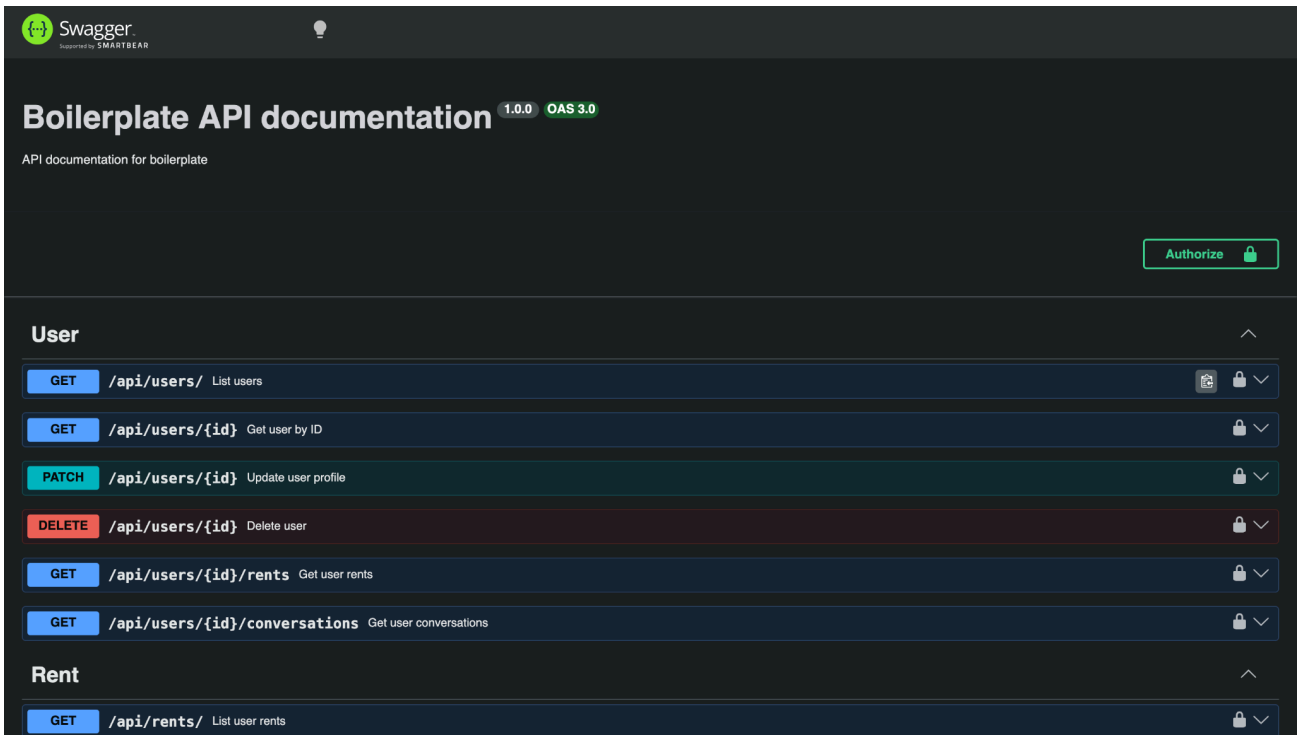
В ходе работы была создана следующая структура на основе приведенного в примере boilerplate:



На основе данных из прошлых практических работ были созданы соответствующие модели. Например, для пользователя:

```
import { Entity, Column, PrimaryGeneratedColumn, BaseEntity, OneToMany } from 'typeorm';
import { Accommodation } from './accommodation.entity';
import { Rent } from './rent.entity';
import { Message } from './message.entity';

export enum Role {
    ADMIN = 'admin',
    USER = 'user'
}

@Entity()
export class User extends BaseEntity {
    @PrimaryGeneratedColumn({ type: 'int' })
    id: number;

    @Column({ type: 'enum', enum: Role, default: Role.USER })
    role: Role;

    @Column({ type: 'varchar', length: 100 })
    first_name: string;

    @Column({ type: 'varchar', length: 100, nullable: true })
    middle_name: string;

    @Column({ type: 'varchar', length: 100 })
    last_name: string;

    @Column({ type: 'varchar', length: 300, unique: true })
    email: string;

    @Column({ type: 'varchar', length: 150 })
    password: string;

    @Column({ type: 'boolean', default: false })
    is_verified: boolean;

    @Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP' })
    created_at: Date;

    @Column({ type: 'timestamp', default: () => 'CURRENT_TIMESTAMP', onUpdate: 'CURRENT_TIMESTAMP' })
    updated_at: Date;

    @OneToMany(() => Accommodation, accommodation => accommodation.landlord)
    accommodations: Accommodation[];

    @OneToMany(() => Rent, rent => rent.tenant)
    rents_as_tenant: Rent[];

    @OneToMany(() => Rent, rent => rent.landlord)
    rents_as_landlord: Rent[];

    @OneToMany(() => Message, message => message.sender)
    sent_messages: Message[];

    @OneToMany(() => Message, message => message.receiver)
    received_messages: Message[];
}
```

Помимо этого, работу можно было протестировать через Swagger:



Вывод

В ходе работы было создано полностью рабочее API согласно варианту про недвижимость.